

rENM Framework User Manual

John L. Schnase

2026-04-18

Table of contents

1	Introduction	3
2	The rENM Workflow	5
3	Configuring the Host Environment	6
3.1	Creating the project directory	6
3.2	Installing the sample data set	6
4	Configuring the RStudio Environment	7
4.1	Setting the project directory location	7
4.2	Installing the framework packages	8
4.3	Loading the rENM framework modules	8
5	Performing an rENM Analysis	8
5.1	Viewing installed species data	9
5.2	Setting the species banding code	9
6	Data Assembly	9
6.1	Occurrence data	9
	- Gathering eBird records	9
	- Removing duplicate records	10
	- Spatial thinning	10
	- Limiting the record count	10
	- Wrapping up occurrence data prep	11
6.2	Environmental data	11
	- Setting the spatial extent	11
	- Assembling MERRA variables	11

7	Time Series Construction	12
7.1	Staging occurrence records	12
7.2	Staging environmental variables	13
	- Option 1 - Simple staging	13
	- Option 2 - Staging with down selection	13
7.3	Reducing covariance	14
7.4	Computing the time series	14
8	Time Series Analysis	15
8.1	Suitability trend analysis	15
	- Finding the suitability trend	15
	- Finding overall trend percentages	15
	- Finding range trend percentages	15
	- Finding state-level suitability trends	15
8.2	Centroid trend analysis	16
	- Finding temporal trends	16
	- Finding bioclimatic velocity	16
8.3	Variable trend analysis	16
	- Gathering ranked variables	16
	- Summarizing variable contributions	17
8.4	Experimental metrics	17
	- Bioclimatic acceleration	17
	- Hot spot analysis	17
9	GenAI Analysis (In development)	18
9.1	Suitability trend analysis	18
9.2	Environmental structure analysis	18
9.3	rENM-based suitability projection	18
9.4	rENM-based conservation planning	18
10	Report Generation	18
10.1	Assemble suitability results	18
10.2	Assemble variable results	19
10.3	Assemble centroid results	19
10.4	Assemble final report	20

1 Introduction

The rENM framework provides a modular collection of R packages for fitting retrospective ecological niche models (rENMs). It combines historical occurrence records and environmental data to estimate the probabilities of suitable climatic conditions occurring across a study area and create time series that can reveal the spatiotemporal dynamics of suitable conditions for various species over time. These historical patterns can shed light on biological questions and the present and future conservation status of a species. While similar studies often rely solely on temperature and precipitation to represent changing climatic conditions, the rENM framework uses a combination of time-specific macroclimatic and microclimatic drivers in its models.

rENMtools is adaptable to many type of organisms; however, it is currently designed to model climate change influences on North American bird species. Its capabilities span the past 45 years and draw on the citizen science observations contained in Cornell Lab’s eBird collections and environmental data from NASA’s Earth system models.

Additional information can be found in the following publications and the project’s GitHub repository at: <https://github.com/rENM-Framework>.

- Schnase, J.L., M.L. Carroll, R.L. Gill, G.S. Tamkin, J. Li, S.L. Strong, T.P. Maxwell, M.E. Aronne, and C.S. Spradlin. Toward a Monte Carlo approach to selecting climate variables in MaxEnt. PLoS One. 2021. Mar 3;16(3):e0237208. [doi:10.1371/journal.pone.0237208](https://doi.org/10.1371/journal.pone.0237208). PMID: 33657125; PMCID: PMC7928495.
- Schnase, J.L. and Carroll, M.L. 2022. Automatic variable selection in ecological niche modeling: a case study using Cassin’s Sparrow (*Peucaea cassinii*). PLoS One. 2022 Jan 21;17(1):e0257502. [doi: 10.1371/journal.pone.0257502](https://doi.org/10.1371/journal.pone.0257502). PMID: 35061658; PMCID: PMC8782318.
- Schnase, J.L., and M.L. Carroll. 2023. “The MMX Toolkit: High-Performance, Reanalysis-Based Climatic Suitability Modeling to Advance Avian Conservation.” In Proceedings of the 2023 Conference on Big Data from Space (BiDS’23): 6-9 November 2023, edited by P. Soille, S. Lumnitz, and S. Albani, 299–303. Austrian Center, Vienna: Publications Office of the European. <https://doi.org/10.2760/46796>.
- Schnase, J. L., M. L. Carroll, P. M. Montesano, and V. A. Seamster. 2024. Complex changes in climatic suitability for Cassin’s Sparrow (*Peucaea cassinii*) revealed by retrospective ecological niche modeling. Journal of Field Ornithology 95(1):9. <https://doi.org/10.5751/JFO-00432-950109>
- Schnase, J. L., M. L. Carroll, P. M. Montesano, and V. A. Seamster. 2025. Shifts in breeding phenology for Cassin’s Sparrow (*Peucaea cassinii*) over four decades. Journal of Field Ornithology 96(3):3. <https://doi.org/10.5751/JFO-00691-960303>

- Schnase, J. L., M. L. Carroll, P. M. Montesano, and V. A. Seamster. 2026. Shifts in seasonal climatic suitability for Cassin's Sparrow (*Peucaea cassinii*) over four decades. *The Southwestern Naturalist*, 70(1):1-17. <https://doi.org/10.1894/0038-4909-70.1.7>

This manual guides the user through the rENM modeling and analysis process. Additional information about each of the functions used in this workflow can be found in the various packages' documentation.

Please note that this is very much a research platform in active development. The rENM framework is being made available for educational purposes only and is shared without technical support of any kind. See the accompany licenses for additional detail. Feedback and further contributions to this effort are welcome.

Thanks for your interest. And good luck!

John Schnase / rENM.Framework@gmail.com

2 The rENM Workflow

3 Configuring the Host Environment

The most straightforward way of configuring your local computing environment is to perform these first steps through a terminal window. I work in a MacOS world using the bash shell, so that's what these examples are based on.

The basic filesystem structure for working with the rENM framework is shown below. From a base project directory you'll have a data directory containing input occurrence records and environmental predictors. In this initial set up, I'll show you how to install an example data set and set a symbolic link to that sample collection. rENM analysis runs will be placed in a *runs* subdirectory within the project's base directory.

```
PROJECT_DIRECTORY/  
|____ data -> (symbolic link to example_data)  
|____ example_data/...  
|____ runs/...
```

You only need to configure your host environment once. After this initial configuration, you're ready to move into the RStudio environment to complete the rENM framework set up. I use *rENMtest* as the project directory name in these examples, but you'll of course use one of your own choosing. I use *~/rENM* in my own work.

3.1 Creating the project directory

You'll first need to create the base project directory and the *runs* subdirectory. Open a terminal window and run the following commands:

```
#!/usr/bin/env bash  
  
# set project directory name  
PROJECT_DIRECTORY="$HOME/rENMtest"  
  
# create initial directory structure  
mkdir -p "$PROJECT_DIRECTORY"  
mkdir -p "$PROJECT_DIRECTORY/runs"
```

3.2 Installing the sample data set

Next, you'll want to install the example data set. Make sure your current working directory is the project base directory you just created, then run the following commands from a terminal window:

```
#!/usr/bin/env bash

# set project directory name (same as above ...)
PROJECT_DIRECTORY="$HOME/rENMtest"
cd ${PROJECT_DIRECTORY}

# download and unzip example data set
DATA_URL="https://storage.googleapis.com/renm_data/example_data.zip"
ZIP_FILE="example_data.zip"
curl -L -o "${ZIP_FILE}" "${DATA_URL}"
unzip -o "${ZIP_FILE}"

# create symbolic link
ln -sfn example_data data

# clean up the installation
rm -rf __MACOSX
rm -f "${ZIP_FILE}"
```

4 Configuring the RStudio Environment

4.1 Setting the project directory location

We begin by letting the RStudio environment and rENM framework know the location of the base project directory you've set up on your local host machine in the previous steps. The following command sets this in the *.Renviron* file. You only need to do this step the first time you set up the environment, so run these commands once, then leave them commented out going forward. The base directory name in this command obviously needs to be the same as the one you set up in the previous steps.

```
# # set the location of the project directory
# write(
#   paste0('RENM_PROJECT_DIR=', file.path(Sys.getenv("HOME"), "rENMtest")),
#   file = "~/Renviron",
#   append = TRUE
# )
# s
# # reload (or restart R)
# readRenviron("~/Renviron")
# Sys.getenv("RENM_PROJECT_DIR")
```

4.2 Installing the framework packages

You'll need to configure your local R/RStudio environment by downloading and installing the rENM framework's collection of module packages. You can get these from the GitHub repository as shown below. There are six modules in the collection. You should only need to install these once. Note that the *rENM.core* package is required by all the other framework modules.

```
# # install rENM framework packages from the GitHub repository
# install.packages("remotes")
# remotes::install_github("rENM-Framework/rENM.core")
# remotes::install_github("rENM-Framework/rENM.data")
# remotes::install_github("rENM-Framework/rENM.model")
# remotes::install_github("rENM-Framework/rENM.analysis")
# remotes::install_github("rENM-Framework/rENM.reports")
# remotes::install_github("rENM-Framework/rENM.ai")
```

4.3 Loading the rENM framework modules

Now, load the installed rENM framework packages.

```
# load the installed rENM framework packages
library(rENM.core)
library(rENM.data)
library(rENM.model)
library(rENM.analysis)
library(rENM.ai)
library(rENM.reports)
```

5 Performing an rENM Analysis

We'll use a four-character banding code (alpha code) to identify a species to be studied. To demonstrate rENMtools' capabilities, we provide data for Brown-capped Rosy-Finch (*Leucosticte australis*) (BCRF), Cassin's Sparrow (*Peucaea cassinii*) (CASP), and Greater Roadrunner (*Geococcyx californianus*) (GRRO) in the base data collections. These three species, respectively, provide examples of small, medium, and large range distributions and occurrence record data sets.

5.1 Viewing installed species data

You can see the species currently available for analysis using the following function. The four-letter ALPHA.CODE is what will be used to identify the species being studied in the step that follows.

```
# show available species
rENM.core::show_species()
```

5.2 Setting the species banding code

```
# set four-letter banding code of a bird species
alpha_code <- "BCRF"
# alpha_code <- "CASP"
# alpha_code <- "GRR0"
```

6 Data Assembly

6.1 Occurrence data

- Gathering eBird records

The first step in performing an rENM time series analysis is to gather and format the input occurrence data and environmental variables you will use in the analysis. We begin by extracting occurrence records for a single species from a downloaded eBird Basic Dataset (EBD) text file in base data collection.

The function described below reads the raw EBD text file, filters records with valid geographic coordinates, writes a clean CSV copy to the run directory, and divides the occurrence data into 5-year temporal bins spanning 1980 through 2020. Each bin is saved as a CSV file in the run's *RENM_PROJECT_DIR*/*<alpha_code>*/*_occs/tmp/* directory for further processing. Use the *show_species()* helper function to see the complete list of available species in rENMtools' base data collection.

```
# extract ebird occurrence records from the base collection
rENM.data::get_ebird_occurrences(alpha_code)
```

- Removing duplicate records

Next, we'll remove duplicate records in the `<alpha_code>/_occs/tmp/` run collection files.

```
# de-duplicate occurrence records
rENM.data::remove_duplicate_occurrences(alpha_code)
```

- Spatial thinning

We now have the option of spatially thinning the occurrence records. Thinning reduces spatial clustering by enforcing a minimum distance between occurrence points, helping to mitigate spatial sampling biases.

For our work with Cassin's Sparrow, we generally thinned records to a 8 km (~5 mile) radius to avoid double counting the same individual bird. You can ignore this thinning step altogether or set your preferred distance by setting the `thin_distance` parameter as shown below.

This thinning step can take some time. The default is to thin the bins sequentially. As shown below, you have the option of running the thinning step as a parallel process, which can save time. However, if you have doubts about doing this on your machine, go with the default sequential thinning operation. Comment out the function you decide not to use.

```
# set thinning distance (radius in km)
thin_distance <- 1

# thin records in the _occs/tmp/ run collection [sequential version]
# thin_occurrences(alpha_code, thin_distance)

# thin records in the _occs/tmp/ run collection [parallel version]
rENM.data::thin_occurrences2(alpha_code, thin_distance)
```

- Limiting the record count

There may be times when you'd like to set an upper bound on the number of occurrence records used in each bin of the rENM time series. This sometimes is done to reduce record densities that can diminish model performance. This is an optional step.

```
# set upper bound on the number of occurrence records
record_count <- 250

# thin records to upper bound
rENM.data::limit_record_count(alpha_code, record_count)
```

- Wrapping up occurrence data prep

We finish setting up occurrence data by moving the prepared per-bin occurrence files from the temporary `<alpha_code>/_occs/tmp/` directory into the main `<alpha_code>/_occs/` directory for our species run. After copying the files, the temporary directory is removed. This is the final cleanup step before preparing environmental data.

```
# move filtered records to the <alpha_code>/_occs/ run collection and tidy up
rENM.data::tidy_occurrences(alpha_code)
```

6.2 Environmental data

- Setting the spatial extent

Preparing environmental data for use in these models involves several steps, some of which are optional. We begin by establishing a spatial extent for the models that we'll be creating. This is a required step, but `rENMtools` provides three options for doing this: model extent can be calculated using the species' eBird occurrence records, model extent can be calculated using the species' USGS GAP range, or model extent can be set manually. Select your preferred approach by commenting out the options you don't want. Run `?set_extent` in the RStudio console to see details on how to manually set function parameters.

```
# set model extent using the usgs gap range
rENM.data::find_range_extent(alpha_code)

# set model extent using ebird occurrences records
# rENM.data::find_occurrence_extent(alpha_code)

# manually set the model extent
# rENM.data::set_extent(alpha_code)
```

- Assembling MERRA variables

We're now ready to move environmental variables from the base data collection into the species' run directory. The first step in this process is to choose which MERRA-2 or MERRAclim-2 variables in the base collection you'd like to use:

- Run the `show_variables()` utility from the RStudio console to see the complete list of available variables, then assign selected variables to the `m2_vars` and `mc_vars` parameters as shown below. NULL options for either means ALL of the available variables of that type will be used. The sample base collection contains the variables used in the Schnase

et al. (2026) paper described above. Additional information is provided in the package documentation.

```
# MERRA-2 microclimatic variables
# m2_vars = c("evpintr", "evpsoil", "prevtot", "qv2m",
#             "speed", "swgnt", "tautot")

# MERRAclim-2 macroclimatic variables
# mc_vars = c("bio3", "bio5", "bio8", "bio13",
#             "bio14", "bio15", "bio18")
```

- Next, we extract the selected MERRA variables from the base collection. The function below does the extraction, crops each variable to our spatial extent, and saves the variables in the run's `<alpha_code>/_vars/` directory for further processing.

```
# extract merra variables and move to the run directory
rENM.data::get_merra_variables(
  alpha_code = alpha_code,
  m2_vars = NULL,
  mc_vars = NULL)
```

7 Time Series Construction

7.1 Staging occurrence records

Having completed preparing the input data, we're now ready to create an rENM time series. To do this, we first move the filtered and refined occurrence records from the `<alpha_code>/_occs/` collection to the `<alpha_code>/TimeSeries/` directory.

The function described here creates the destination *TimeSeries* directory and sets up 5-year bins labeled by year. Occurrence record files are moved into an *occs* subdirectory within the newly-created *TimeSeries* directory. The destination for these moves is therefore `<alpha_code>/TimeSeries/<year>/occs/` where `<year>` runs from 1980 to 2020 in 5-year increments.

This is confusing to describe, but it becomes obvious after you run the function and look at the `<alpha_code>/TimeSeries/` directory structure.

```
# move prepared occurrence files from the <alpha_code>/_occs/ directory
# to 5-year bins within the <alpha_code>/TimeSeries/occs/ directory
rENM.model::stage_occurrences(alpha_code)
```

7.2 Staging environmental variables

We now do the same thing with our environmental variables. There are two options for doing this, and you should only choose one:

- *Option 1 - Simple staging.* All of the cropped variables saved in the `<alpha_code>/_vars/` collection are moved to the `<alpha_code>/TimeSeries/` directory without further modification of any kind.
- *Option 2 - Staging with down selection.* The Monte Carlo-based down selection method described in Schnase et al. (2022) is first applied to the variables saved in the `<alpha_code>/_vars/` collection. This step identifies the top performing variables in the collection. This down selected set of variables is then moved into the time series for further processing.

- Option 1 - Simple staging

The function described here performs simple staging: it moves all of the cropped environmental predictors from the `<alpha_code>/_vars/` collection to the `<alpha_code>/TimeSeries/` directory without further modification of any kind.

The files are moved into a *vars* subdirectory within the *TimeSeries* directory that was created by the previous operation. The destination for these moves is therefore `<alpha_code>/TimeSeries/<year>/vars/` where `<year>` runs from 1980 to 2020 in 5-year increments.

```
# move all the cropped environmental variables from the <alpha_code>/_vars/  
# directory to 5-year bins within the <alpha_code>/TimeSeries/vars/ directory  
# rENM.model::stage_all_variables(alpha_code)
```

- Option 2 - Staging with down selection

[In progress ...]

```
# down select variables (maxent version)  
# rENM.model::screen_by_convergence1(alpha_code)  
  
# down select variables (maxnet version)  
rENM.model::screen_by_convergence2(alpha_code)
```

```
# move the down selected, cropped environmental variables from the
# the <alpha_code>/_vars/ directory to 5-year bins within the
# <alpha_code/TimeSeries/vars/ directory
rENM.model::stage_screened_variables(alpha_code)
```

7.3 Reducing covariance

This optional step reduces covariance in the staged environmental variables, records results, and retains only non-correlated variables for further processing. The default settings here perform a “light touch” collinearity reduction using variance inflation factors (VIF) and pairwise correlation thresholds. A relaxed VIF threshold (10) and high correlation threshold (0.95) are used to remove only extreme redundancy while preserving alternative representations of environmental gradients, thus ensuring that key dimensions of environmental structure identified during screening were retained in the final model.

```
# reduce covariance among staged variables
# rENM.model::reduce_covariance(alpha_code)
```

7.4 Computing the time series

We’re now ready to create a time series of ensemble models using the staged occurrence records and staged variables. There’s a lot of hidden detail here. The *create_timeseries()* function shown in the code block below makes calls on the *create_ensemble_model()* function to build up the time series. These calls run in parallel.

In the default configuration, we’re find the unweighted average of five model types: maxnet, random forest (rf), boosted regression tree (brt), generalized linear model (glm), and and multivariate adaptive regression splines (mars). Browse the online documentation using *?create_ensemble_model* for additional information.

```
# create an rENM time series
rENM.model::create_timeseries(alpha_code)
```

8 Time Series Analysis

8.1 Suitability trend analysis

- Finding the suitability trend

This required step computes per-cell Theil-Sen trends from the time series of predicted suitability rasters and derives Mann-Kendall statistics (p and Z) to assess temporal trends in climatic suitability. This trend raster becomes the basis further analyses.

```
# compute the thiel-sen trend across the time series
rENM.analysis::find_suitability_trend(alpha_code)
```

- Finding overall trend percentages

This is the first of several optional looks at the suitability trends that we've derived from our rENM time series. The function below computes cell-wise trend sign statistics from the suitability trend raster we've just calculated and summarizes proportions of positive, negative, and zero values across the full modeled extent.

```
# find proportions of pos, neg, and zero values across modeled extent
rENM.analysis::find_trend_percentages(alpha_code)
```

- Finding range trend percentages

The function below computes cell-wise trend sign statistics from the suitability trend raster we've just calculated and summarizes proportions of positive, negative, and zero values across the species' USGS GAP range.

```
# find proportions of pos, neg, and zero values across USGS GAP range
rENM.analysis::find_range_change_percentages(alpha_code)
```

- Finding state-level suitability trends

We can now perform a per-state analysis of climatic suitability trends based on the species' USGS GAP range. The function below does the analysis and generates a map showing spatial patterns and state-level metrics for the state that encompass the species' GAP range.

```
# create state-level suitability trend analysis and map for a species
rENM.analysis::create_state_trend_analysis(alpha_code)
```

8.2 Centroid trend analysis

- Finding temporal trends

Another way of looking at climatic suitability trends for the species is by analyzing spatial shifts in the weighted centroid for suitability over time. This function fits Bayesian linear trends to the species' annual weighted centroids for suitability (latitude and longitude) and writes plots and summaries to the species run directory.

```
# analyze temporal trends in suitability centroids
rENM.analysis::analyze_weighted_centroids(alpha_code)
```

- Finding bioclimatic velocity

Historical patterns of centroid movement also can be used to determine the bearing, distance, and velocity of suitability shifts. We calculate these values and save them to our run directory in the following step.

```
# finds bioclimatic verlocity from weighted centroids
rENM.analysis::find_bioclimatic_velocity(alpha_code)
```

To aid visualizing long-term shifts, this function creates and saves our suitability trend map with an overlay of the centroid displacement vector.

```
# plot and save suitability trend map with centroids
rENM.analysis::save_trend_plot_with_centroids(alpha_code)
```

8.3 Variable trend analysis

- Gathering ranked variables

Consolidate ranked variable importance outputs across time-series runs into a single summary dataset for a species. This step produces a unified CSV file that can be used in further variable analysis.


```
# gather variable rankings
rENM.analysis::gather_variable_contributions(alpha_code)
```

- Summarizing variable contributions

Reads a species-specific CSV of variable contributions across the time series, selects variables by average relative contribution, and produces a complete suite of visualizations and statistics.

```
# summarize variable contributions
rENM.analysis::summarize_variable_contributions(alpha_code)
```

8.4 Experimental metrics

- Bioclimatic acceleration

The following function wraps *find_suitability_change_trend()* and *plot_suitability_change_trend()* to create a map that essentially shows areas of accelerating or decelerating trends in climatic suitability. This is still experimental, but it's an interesting perspective on the spatial and temporal dynamics of historical climatic influences on suitability.

```
# create a climatic suitability trend acceleration/deceleration map
rENM.analysis::create_suitability_change_map(alpha_code)
```

- Hot spot analysis

In this metric we find areas where the baseline trend in climatic suitability is *decreasing at an increasing rate*. This function creates a map showing those areas. The question is whether such a metric could be a useful indicator of increased climate change vulnerability ...

```
# create a hot spot map
rENM.analysis::create_hot_spot_map(alpha_code)
```

9 GenAI Analysis (In development)

9.1 Suitability trend analysis

This experimental collection of tools draws on the capabilities of GenAI to do suitability trend analysis. In a first step, the function collects suitability-trend rasters, summary tables, and species metadata for a species, then builds an AI-ready package together with a prompt template. In a second step, the package is submitted to OpenAI's ChatGPT gpt-5.1 LLM for analysis. Finally, the completed analysis is converted to a PDF document.

```
# build a data package for submission to ChatGPT
rENM.ai::assemble_suitability_ai_package(alpha_code)

# submit the package using the OpenAI Responses API
rENM.ai::submit_suitability_ai_package(alpha_code)

# render the returned .docx file into a .pdf file
rENM.ai::render_ai_docs(alpha_code)
```

9.2 Environmental structure analysis

9.3 rENM-based suitability projection

9.4 rENM-based conservation planning

10 Report Generation

The following set of functions gathers key outputs from the analysis steps above and creates summary maps, tables, and reports. It's important to note these capabilities are grouped together in the *rENM.reports* R package, which has to be installed and loaded before you can run these steps. Because they draw on products created by earlier steps in the rENM workflow, you have to have all the steps above before moving on the summaries and report generation.

10.1 Assemble suitability results

```
# gather time series suitability maps
rENM.reports::gather_suitability_maps(alpha_code)

# gather range maps
```

```

rENM.reports::gather_range_maps(alpha_code)

# gather suitability trend and hot spot stats
rENM.reports::gather_suitability_trend_stats(alpha_code)

# create a suitability trend summary table
rENM.reports::create_suitability_trend_summary_table(alpha_code)

# create a suitability time series summary page
rENM.reports::assemble_suitability_timeseries_page(alpha_code)

# create a range time series summary page
rENM.reports::assemble_range_timeseries_page(alpha_code)

# create a suitability trends, acceleration/decleration page
rENM.reports::assemble_suitability_trends_page(alpha_code)

# create a state trends and state hot spot page
rENM.reports::assemble_state_trends_page(alpha_code)

```

10.2 Assemble variable results

```

# create variable trend summary table
rENM.reports::create_variable_trend_summary_table(alpha_code)

# create variable trends page
rENM.reports::assemble_variable_trends_page(alpha_code)

# gather trend and rate-of-change maps for the top 10 variables
rENM.reports::gather_top_variable_trend_maps(alpha_code)

# create summary page for the top 10 variables
rENM.reports::assemble_variable_trend_maps_page(alpha_code)

```

10.3 Assemble centroid results

```

# create centroid trend summary table
rENM.reports::create_centroid_trend_summary_table(alpha_code)

```

```
# create a suitability trends with centroids page
rENM.reports::assemble_centroid_trends_page(alpha_code)
```

10.4 Assemble final report

```
# create a final report comprising all summary pages
rENM.reports::assemble_final_report(alpha_code)

# these are the default pages, edit as needed ...
# rENM.reports::assemble_final_report(alpha_code,
#                                     pages = c("Suitability-Trend-Analysis",
#                                     "Suitability-Trends",
#                                     "State-Trends",
#                                     "Centroid-Trends",
#                                     "Suitability-TimeSeries",
#                                     "Range-TimeSeries",
#                                     "Variable-Trends",
#                                     "Variable-Trend-Maps1",
#                                     "Variable-Trend-Maps2"))
```